

Q1. Verifying Kernel Sharing

Create a Container for **Ubuntu** image and Check the Kernel Version

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con_ubuntu ubuntu bash
root@4147b81a56d9:/# uname -r
6.14.0-1016-aws
```

Create a Container for **Alpine** image and Check the Kernel Version

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con_alpine alpine sh
/ # uname -r
6.14.0-1016-aws
```

Host - Check Kernel Version

```
ubuntu@ip-10-0-1-18:~$ uname -r
6.14.0-1016-aws
```

Observation

- When create a container for **Ubuntu image**, check the kernel version get the output as '**6.14.0-1016-aws**'.
- When create a container for **Alpine image**, check the kernel version get the output as '**6.14.0-1016-aws**'.
- When check the kernel version of the **Host** and get the output as '**6.14.0-1016-aws**'.
- So, we can come to a conclusion all docker containers are using the same kernel as the host.

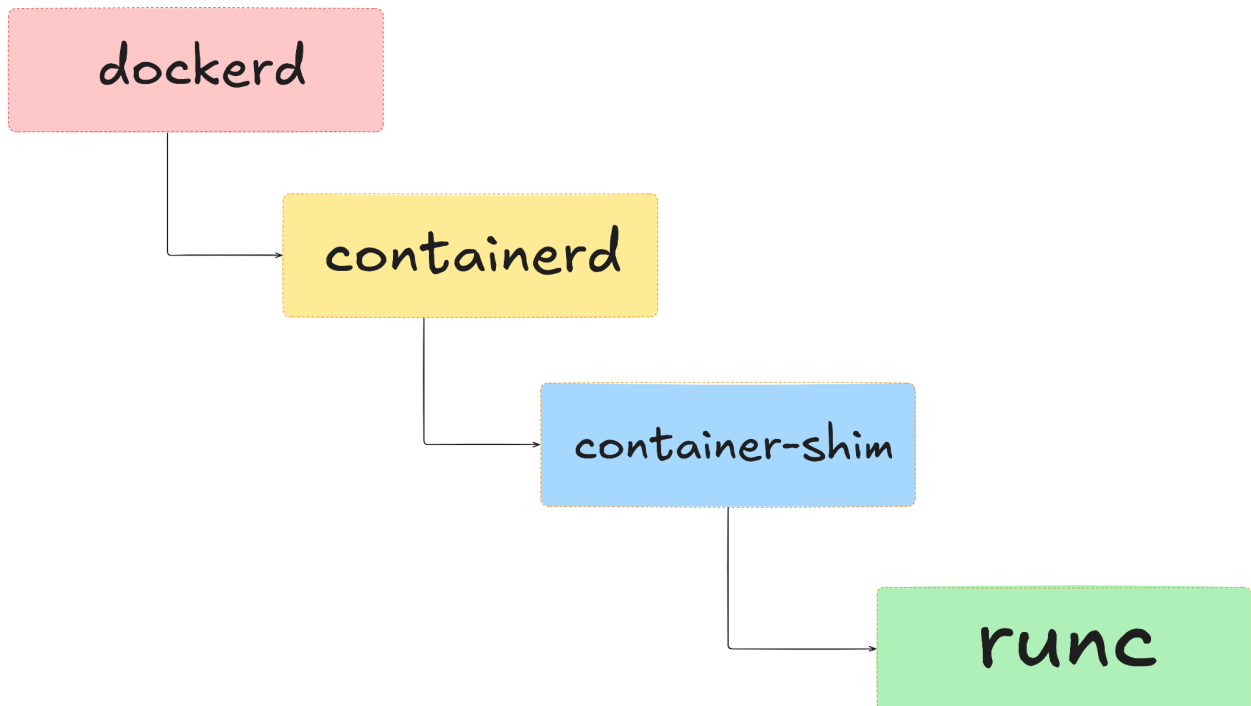
Based on our observation we can clearly see that containers don't have a dedicated kernel for themselves they are sharing the kernel of the host. But, if it's a VM; it definitely has its own kernel. So, we can state that docker containers are not VMs.

Q2. Process and PID Mapping

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
root@5938e90d49a1:/# echo $$
1
```

```
ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con
2552
ubuntu@ip-10-0-1-18:~$ pstree -ps 2552
systemd(1)──containerd-shim(2529)──bash(2552)
```

This is how container creation process works



Q3. Exploring Namespace Isolation

Container 1 Creation

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
root@5938e90d49a1:/# echo $$
1
```

Container 1 Inspection

```
ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con
1508
ubuntu@ip-10-0-1-18:~$ sudo ls -l /proc/1508/ns/
total 0
lrwxrwxrwx 1 root root 0 Nov  6 15:38 cgroup -> 'cgroup:[4026532220]'
lrwxrwxrwx 1 root root 0 Nov  6 15:38 ipc -> 'ipc:[4026532218]'
lrwxrwxrwx 1 root root 0 Nov  6 15:29 mnt -> 'mnt:[4026532216]'
lrwxrwxrwx 1 root root 0 Nov  6 15:29 net -> 'net:[4026532221]'
lrwxrwxrwx 1 root root 0 Nov  6 15:38 pid -> 'pid:[4026532219]'
lrwxrwxrwx 1 root root 0 Nov  6 15:46 pid_for_children -> 'pid:[4026532219]'
lrwxrwxrwx 1 root root 0 Nov  6 15:38 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Nov  6 15:46 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Nov  6 15:38 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 Nov  6 15:38 uts -> 'uts:[4026532217]'
```

Container 2 Creation

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con1 ubuntu bash
root@998645704a76:/# echo $$
1
```

Container 2 Inspection

```
ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con1
1826
ubuntu@ip-10-0-1-18:~$ sudo ls -l /proc/1826/ns/
total 0
lrwxrwxrwx 1 root root 0 Nov  6 15:49 cgroup -> 'cgroup:[4026532295]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 ipc -> 'ipc:[4026532293]'
lrwxrwxrwx 1 root root 0 Nov  6 15:48 mnt -> 'mnt:[4026532291]'
lrwxrwxrwx 1 root root 0 Nov  6 15:48 net -> 'net:[4026532296]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 pid -> 'pid:[4026532294]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 pid_for_children -> 'pid:[4026532294]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 Nov  6 15:49 uts -> 'uts:[4026532292]'
```

Namespace	Container 1	Container 2	Shared / Unique
CGROUP	4026532220	4026532295	Unique
IPC	4026532218	4026532293	Unique
PID	4026532219	4026532294	Unique
NET	4026532221	4026532296	Unique
MNT	4026532216	4026532291	Unique
USER	4026531837	4026531837	Shared
UTS	4026532217	4026532292	Unique

So, basically unique ns for isolation and shared means the container have communication / control with each other.

Observation

When we are creating 2 containers for the **Ubuntu image** , inspect them for the name spaces; we can see that some items like PID,NET are **Unique** and USER is **Shared**.

Q4. Observing New Namespace Creation

Get dockerd's Pid

```
ubuntu@ip-10-0-1-18:~$ ps -ef | grep dockerd
root      728      1  0 13:26 ?        00:00:00 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Trace the process

```
ubuntu@ip-10-0-1-18:~$ sudo strace -fp 728 -e trace=clone,unshare,setns
strace: Process 728 attached with 12 threads
```

Create a new container

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it ubuntu bash
```

Now see the trace again

```
ubuntu@ip-10-0-1-18:~$ sudo strace -fp 728 -e trace=clone,unshare,setns
strace: Process 728 attached with 12 threads
[pid 1425] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 3448] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 3448] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 1425] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 1425] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 757] setns(31, CLONE_NEWNET) = 0
[pid 757] setns(32, CLONE_NEWNET) = 0
[pid 757] setns(31, CLONE_NEWNET) = 0
[pid 757] setns(32, CLONE_NEWNET) = 0
[pid 757] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 3522 attached
, parent_tid=[38]) = 3522
[pid 757] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 3522] +++ exited with 1 +++
[pid 757] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3522, si_uid=0, si_status=1, si_etime=0, si_stime=0} ---
[pid 778] clone(child_stack=NULL, flags=CLONE_VM|CLONE_PIDFD|CLONE_VFORK|SIGCHLDstrace: Process 3524 attached
, parent_tid=[38]) = 3524
[pid 3524] +++ exited with 0 +++
[pid 1424] --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=3524, si_uid=0, si_status=0, si_etime=0, si_stime=0} ---
[pid 1424] setns(32, CLONE_NEWNET) = 0
[pid 1424] setns(34, CLONE_NEWNET) = 0
[pid 1424] setns(31, CLONE_NEWNET) = 0
[pid 1424] setns(32, CLONE_NEWNET) = 0
[pid 1424] setns(32, CLONE_NEWNET) = 0
[pid 1424] setns(34, CLONE_NEWNET) = 0
[pid 778] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 778] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 778] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 1424] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
[pid 757] --- SIGURG {si_signo=SIGURG, si_code=SI_TKILL, si_pid=728, si_uid=0} ---
```

Observation

When we trace the **dockerd** in the beginning there is no tracks. After we command for a container creation in separate terminal we can see the **dockerd** call the **setns()**, **clone()** methods for creating the container.

- **CLONE_VM** : Child process shares same memory space as parent.
- **CLONE_PIDFD** : Creates new PID File Descriptor for tracking.
- **CLONE_VFORK** : Suspend parent until child process execs
- **CLONE_NEWNET** : Create new network ns for child process.

Q5. Investigating cgroup Assignments

Create container with limited resource

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con --cpus="0.5" --memory="100m" ubuntu bash
```

Get PId and locate cgroup path

```
ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con
2409
ubuntu@ip-10-0-1-18:~$ cat /proc/2409/cgroup
0::/system.slice/docker-a21f61d5e85deaf1e73862fe3bf66f1779c68c894d8e604a20e304d5b855ebf5.scope
```

Identify active controllers

```
ubuntu@ip-10-0-1-18:~$ ls /sys/fs/cgroup/system.slice/docker-a21f61d5e85deaf1e73862fe3bf66f1779c68c894d8e604a20e304d5b855ebf5.scope
cgroup.controllers      cpu.pressure            dmabuf.current          hugetlb.2MB.rsvd.current  memory.oom.group         misc.events.local
cgroup.events           cpu.stat               dmabuf.min              hugetlb.2MB.rsvd.max     memory.peak              misc.max
cgroup.freeze          cpu.stat.local         dmabuf.max              hugetlb.current          memory.pressure           misc.peak
cgroup.kill            cpu.uclamp.max         hugetlb.1GB.current     io.pressure              memory.reclaim            pids.current
cgroup.max.depth       cpu.uclamp.min         hugetlb.1GB.events      io.prio.class            memory.stat               pids.events
cgroup.max.descendants   cpu.weight             hugetlb.1GB.events.local io.stat                  memory.swap.current       pids.events.local
cgroup.pressure        cpu.weight.nice        hugetlb.1GB.max         io.weight                 memory.swap.events        pids.max
cgroup.procs           cpuset.cpus            hugetlb.1GB.numa_stat   memory.current            memory.swap.high          pids.peak
cgroup.stat            cpuset.cpus.effective hugetlb.1GB.rsvd.current memory.events             memory.swap.max           rdma.current
cgroup.subtree_control cpuset.cpus.exclusive hugetlb.1GB.rsvd.max     memory.events.local       memory.swap.peak          rdma.max
cgroup.threads         cpuset.cpus.exclusive.effective hugetlb.2MB.current     memory.high                memory.swap.current
cgroup.type            cpuset.cpus.partition hugetlb.2MB.events      memory.low                 memory.zswap.max
cpu.idle               cpuset.mems            hugetlb.2MB.events.local memory.max                  memory.zswap.writeback
cpu.max                cpuset.mems.effective hugetlb.2MB.max         memory.min                  misc.current
cpu.max.burst          dmabuf.current         hugetlb.2MB.numa_stat   memory.numa_stat           misc.events
```

Some active controllers

- cpu - CPU allocation from the host
- memory - Memory allocation from the host
- pids - Limit of the process that container can create
- io - limits the bandwidth for read / write

Check the limits

```
ubuntu@ip-10-0-1-18:~$ cat /sys/fs/cgroup/system.slice/docker-a21f61d5e85deaf1e73862fe3bf66f1779c68c894d8e604a20e304d5b855ebf5.scope/cpu.max
50000 100000
ubuntu@ip-10-0-1-18:~$ cat /sys/fs/cgroup/system.slice/docker-a21f61d5e85deaf1e73862fe3bf66f1779c68c894d8e604a20e304d5b855ebf5.scope/memory.max
104857600
```

For a default container limits be like this

```
ubuntu@ip-10-0-1-18:~$ cat /sys/fs/cgroup/system.slice/docker-786945aa71f419dab1b8c9cf86525ea3aedc21b1155524d73fa1977bd5a633a2.scope/cpu.max
max 100000
ubuntu@ip-10-0-1-18:~$ cat /sys/fs/cgroup/system.slice/docker-786945aa71f419dab1b8c9cf86525ea3aedc21b1155524d73fa1977bd5a633a2.scope/memory.max
max
```

Observation

When checking the limits for the default and limited container

- Default one is memory and cpu is in maximum
- Limited one's memory and cpu is specified size

Q6. Resource Behavior Under Load

For a Normal Container Stress test results be like

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
```

CPU Stress

```
root@116aa45ea695:/# stress --cpu 2 --timeout 60s
stress: info: [161] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [161] successful run completed in 60s
```

1. On Container Stat

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
116aa45ea695	con1	198.41%	26.41MiB / 914.2MiB	2.89%	34.9MB / 188kB	27.7MB / 91.4MB	4

2. On Host Stat

```
top - 19:32:36 up 9 min, 3 users, load average: 1.27, 0.38, 0.13
Tasks: 129 total, 3 running, 126 sleeping, 0 stopped, 0 zombie
%Cpu(s): 99.8 us, 0.2 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 914.2 total, 118.5 free, 417.4 used, 537.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 496.8 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1689	root	20	0	3620	456	456	R	99.7	0.0	0:55.52	stress
1690	root	20	0	3620	456	456	R	99.3	0.0	0:55.25	stress
588	root	20	0	1803400	49528	35592	S	0.3	5.3	0:00.69	containerd
1687	ubuntu	20	0	12368	5844	3668	R	0.3	0.6	0:00.11	top

Memory Stress

```
root@aeb5685b426e:/# stress --vm 2 --vm-bytes 50M --timeout 60s
stress: info: [167] dispatching hogs: 0 cpu, 0 io, 2 vm, 0 hdd
stress: info: [167] successful run completed in 60s
```

1. On Container Stat

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
116aa45ea695	con1	100.28%	193.8MiB / 914.2MiB	21.20%	34.9MB / 188kB	27.7MB / 91.4MB	3

2. On Host Stat

```
top - 19:35:27 up 12 min, 3 users, load average: 0.41, 0.29, 0.12
Tasks: 125 total, 3 running, 122 sleeping, 0 stopped, 0 zombie
%Cpu(s): 9.5 us, 40.3 sy, 0.0 ni, 49.4 id, 0.0 wa, 0.0 hi, 0.0 si, 0.8 st
MiB Mem : 914.2 total, 173.1 free, 519.8 used, 379.8 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 394.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1715	root	20	0	208424	96720	456	R	100.0	10.3	0:24.45	stress
588	root	20	0	1803400	49528	34312	S	0.3	5.3	0:00.99	containerd
737	root	20	0	2048336	81012	53936	S	0.3	8.7	0:00.81	dockerd
1680	root	20	0	1777780	32560	21936	S	0.3	3.5	0:00.39	docker
1	root	20	0	22196	13444	9604	S	0.0	1.4	0:01.36	systemd

For a Limited Container Stress test results be like

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con1 --cpus="0.5" --memory="100m" ubuntu bash
```

CPU Stress

```
root@798b3b9374b6:/# stress --cpu 2 --timeout 60s
stress: info: [161] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [161] successful run completed in 60s
```

1. On Container Stat

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
798b3b9374b6	con1	50.85%	6.492MiB / 100MiB	6.49%	35MB / 178kB	111MB / 92.9MB	4

2. On Host Stat

```
top - 19:42:19 up 18 min, 3 users, load average: 0.07, 0.13, 0.10
Tasks: 129 total, 3 running, 126 sleeping, 0 stopped, 0 zombie
%Cpu(s): 49.6 us, 0.0 sy, 0.0 ni, 49.4 id, 0.0 wa, 0.0 hi, 0.0 si, 1.0 st
MiB Mem : 914.2 total, 137.2 free, 415.0 used, 521.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 499.2 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2032	root	20	0	3620	456	456	R	50.2	0.0	0:14.60	stress
2031	root	20	0	3620	456	456	R	49.8	0.0	0:14.68	stress
2023	root	20	0	1703792	31572	21844	S	0.3	3.4	0:00.08	docker
1	root	20	0	22196	13444	9604	S	0.0	1.4	0:01.42	systemd

Memory Stress

```
root@45526869a293:/# stress --vm 2 --vm-bytes 40M --timeout 60s
stress: info: [174] dispatching hogs: 0 cpu, 0 io, 2 vm, 0 hdd
stress: info: [174] successful run completed in 60s
```

1. On Container Stat

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
32ff621b84ce	con3	50.22%	7.422MiB / 100MiB	7.42%	35MB / 171kB	67.9MB / 91.8MB	3

2. On Host Stat

```
top - 19:46:32 up 23 min, 3 users, load average: 0.41, 0.26, 0.16
Tasks: 128 total, 2 running, 126 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.8 us, 20.4 sy, 0.0 ni, 74.3 id, 0.2 wa, 0.0 hi, 0.0 si, 0.3 st
MiB Mem : 914.2 total, 114.0 free, 501.7 used, 457.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 412.5 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2323	root	20	0	54824	18216	456	R	49.8	1.9	0:11.64	stress
17	root	20	0	0	0	0	I	0.3	0.0	0:00.40	rcu_sched
1	root	20	0	22196	13444	9604	S	0.0	1.4	0:01.47	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.01	kthreadd

Check Logs

```
ubuntu@ip-10-0-1-18:~$ sudo dmesg | grep -i "Killed process"
[ 8818.612156] Out of memory: Killed process 3518 (stress) total-vm:413224kB, anon-rss:319232kB, file-rss:0kB, shmem-rss:0kB, UID:0 pgtables:684kB oom_score_adj:0
[ 8975.165415] Memory cgroup out of memory: Killed process 3651 (stress) total-vm:208424kB, anon-rss:101248kB, file-rss:328kB, shmem-rss:0kB, UID:0 pgtables:248kB oom_score_adj:0
```


Observation

- When we put load for a default container we can see that CPU usage and Memory usage are Higher.
- When we put load for a limited container we can see that CPU usage and Memory usage are in under control which we specified.
- In logs we can see
 1. Process Killed on default Container because of OOM.
 2. Process killed on Limited Container because it required memory more than we allocated.

Q7. Filesystem Layer Analysis

Create a container

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
```

Inspect the container file system

```
ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{json .GraphDriver.Data}}' con | jq
{
  "ID": "0c01b3defd285c683de0c66f0108546328bcb473128719cbff977236a0852d20",
  "LowerDir": "/var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18-init/diff:/var/lib/docker/overlay2/e14a65ae2b9b9b1816bafc36644f125353db3ad8b0278b34d49de5eadf9010fb/diff",
  "MergedDir": "/var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/merged",
  "UpperDir": "/var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff",
  "WorkDir": "/var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/work"
}

ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/
total 16
drwx-x-x--- 3 root root 4096 Nov  7 02:00 e14a65ae2b9b9b1816bafc36644f125353db3ad8b0278b34d49de5eadf9010fb
drwx-x-x--- 5 root root 4096 Nov  7 02:00 e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18
drwx-x-x--- 4 root root 4096 Nov  7 02:00 e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18-init
drwx----- 2 root root 4096 Nov  7 02:09 l
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18
total 20
drwxr-xr-x 2 root root 4096 Nov  7 02:00 diff
-rw-r--r-- 1 root root  26 Nov  7 02:00 link
-rw-r--r-- 1 root root  57 Nov  7 02:00 lower
drwxr-xr-x 1 root root 4096 Nov  7 02:00 merged
drwx----- 3 root root 4096 Nov  7 02:00 work
```

Before Making any Changes

```
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff
total 0
```

Creating a **test.txt** file inside the container

```
root@0c01b3defd28:/# cd /root
root@0c01b3defd28:~# echo 'Hello, World' > test.txt
```

After Making the change

```
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff
total 4
drwx----- 2 root root 4096 Nov  7 02:17 root
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff/root
total 4
-rw-r--r-- 1 root root 13 Nov  7 02:17 test.txt
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff/root/test.txt
-rw-r--r-- 1 root root 13 Nov  7 02:17 /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff/root/test.txt
ubuntu@ip-10-0-1-18:~$ sudo cat /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18/diff/root/test.txt
Hello, World
```

When we stopped the container and inspect the file system host won't remove the **Write Layer / Upper Dir** files

```
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18
total 16
drwxr-xr-x 3 root root 4096 Nov  7 02:00 diff
-rw-r--r-- 1 root root  26 Nov  7 02:00 link
-rw-r--r-- 1 root root  57 Nov  7 02:00 lower
drwx----- 3 root root 4096 Nov  7 02:00 work
```

When we remove the container the files related to this container will be deleted from the host

```
ubuntu@ip-10-0-1-18:~$ sudo docker rm con
con
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18
ls: cannot access '/var/lib/docker/overlay2/e9fc1cd03eeacffb79eca642ec2c3dce3d068627d63308de1dbe9a2ac51fba18': No such file or directory
ubuntu@ip-10-0-1-18:~$ sudo ls -l /var/lib/docker/overlay2/
total 8
drwx--x--- 3 root root 4096 Nov  7 02:00 e14a65ae2b9b9b1816bafc36644f125353db3ad8b0278b34d49de5eadf9010fb
drwx----- 2 root root 4096 Nov  7 02:27 l
```

Observation

- When we inspect the file layers there are 4 layer
 - LowerDir : Base images file (read only layer), which can't be modified
 - UpperDir : Where our changes applied (writable layer)
 - MergedDir : Where the LowerDir + UpperDir result shown, which we witness using container's file system
 - WorkDir : Docker used it for temporary storage used before merging
- Now, we create a file in the container called 'test.txt', it'll appear in the UpperDir folder not on the LowerDir.
- When we stop the container the Files won't get deleted.
- But, if we removed the container then files will be deleted.

Q8. Process Creation Flow Examination

Create a new container

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
4b3ffd8ccb52: Pull complete
Digest: sha256:66460d557b25769b102175144d538d88219c077c678a49af4afca6fbfc1b5252
Status: Downloaded newer image for ubuntu:latest
root@acea3199994d:/# |
```

Get process tree view

```
ubuntu@ip-10-0-1-18:~$ pstree -ap | grep -E 'dockerd|containerd|con'
|-containerd,591
|   |--{containerd},627
|   |--{containerd},631
|   |--{containerd},632
|   |--{containerd},633
|   |--{containerd},634
|   |--{containerd},716
|   |--{containerd},726
|   |--{containerd},1972
|   |--containerd-shim,2529 -namespace moby -id acea3199994daa646bcafbf2d919b64984696c7b36a81d0a261fc55076a9d4ee -address/run/
|       |--{containerd-shim},2530
|       |--{containerd-shim},2531
|       |--{containerd-shim},2532
|       |--{containerd-shim},2533
|       |--{containerd-shim},2534
|       |--{containerd-shim},2535
|       |--{containerd-shim},2536
|       |--{containerd-shim},2537
|       |--{containerd-shim},2558
|       |--{containerd-shim},2574
|   |--dockerd,2259 -H fd:// --containerd=/run/containerd/containerd.sock
|       |--{dockerd},2260
|       |--{dockerd},2261
|       |--{dockerd},2262
|       |--{dockerd},2263
|       |--{dockerd},2265
|       |--{dockerd},2266
|       |--{dockerd},2512
|       |--{dockerd},2517
|       |--{dockerd},2518
|       |--{dockerd},2519
|       |--|-grep,2584 --color=auto -E dockerd|containerd|con
|           |--sudo,2503 docker run -it --name con ubuntu bash
|           |--sudo,2504 docker run -it --name con ubuntu bash
|           |--docker,2505 run -it --name con ubuntu bash
```

Observation

When we create a container and get it's Pid then we get the tree view

Then we clearly see that **systemd** creates the **dockerd**, **containerd**, **container-shim**.

When we create a new container; **dockerd's** threads handle that.

Q9. Comparative Namespace Experiment

Clone() : used to create new process. We can specify which ns are share and unique with flags

setns() : used to join existing ns instead of creating a new one. Does not create new process

Create a Default Container

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con ubuntu bash
```

Get Container Processes

```
root@7a6eb5a72a78:/# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1         0  0  02:46 pts/0        00:00:00 bash
root           9         0  0  03:12 pts/1        00:00:00 bash
root          19         9  0  03:12 pts/1        00:00:00 ps -ef
```

Create a Container which shares host ns

```
ubuntu@ip-10-0-1-18:~$ sudo docker run -it --name con1 --pid=host --network host ubuntu bash
```

Get Container Processes

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	01:45	?	00:00:01	/sbin/init
root	2	0	0	01:45	?	00:00:00	[kthreadd]
root	3	2	0	01:45	?	00:00:00	[pool_workqueue_release]
root	4	2	0	01:45	?	00:00:00	[kworker/R-rcu_gp]
root	5	2	0	01:45	?	00:00:00	[kworker/R-sync_wq]
root	6	2	0	01:45	?	00:00:00	[kworker/R-kvfree_rcu_reclaim]
root	7	2	0	01:45	?	00:00:00	[kworker/R-slub_flushwq]
root	8	2	0	01:45	?	00:00:00	[kworker/R-netns]
root	9	2	0	01:45	?	00:00:00	[kworker/0:0-events]
root	10	2	0	01:45	?	00:00:00	[kworker/0:0H-events_highpri]
root	13	2	0	01:45	?	00:00:00	[kworker/R-mm_percpu_wq]
root	14	2	0	01:45	?	00:00:00	[rcu_tasks_rude_kthread]
root	15	2	0	01:45	?	00:00:00	[rcu_tasks_trace_kthread]
root	16	2	0	01:45	?	00:00:00	[ksoftirqd/0]
root	17	2	0	01:45	?	00:00:00	[rcu_sched]
root	18	2	0	01:45	?	00:00:00	[rcu_exp_par_gp_kthread_worker/0]
root	19	2	0	01:45	?	00:00:00	[rcu_exp_gp_kthread_worker]
root	20	2	0	01:45	?	00:00:00	[migration/0]
root	21	2	0	01:45	?	00:00:00	[idle_inject/0]
root	22	2	0	01:45	?	00:00:00	[cpuhp/0]
root	23	2	0	01:45	?	00:00:00	[cpuhp/1]
root	24	2	0	01:45	?	00:00:00	[idle_inject/1]
root	25	2	0	01:45	?	00:00:00	[migration/1]
root	26	2	0	01:45	?	00:00:00	[ksoftirqd/1]
root	28	2	0	01:45	?	00:00:00	[kworker/1:0H-events_highpri]
root	29	2	0	01:45	?	00:00:00	[kdevtmpfs]
root	30	2	0	01:45	?	00:00:00	[kworker/R-inet_frag_wq]
root	31	2	0	01:45	?	00:00:00	[kauditd]
root	32	2	0	01:45	?	00:00:00	[khungtaskd]
root	34	2	0	01:45	?	00:00:00	[oom_reaper]
root	35	2	0	01:45	?	00:00:00	[kworker/u8:2-events_unbound]
root	36	2	0	01:45	?	00:00:00	[kworker/R-writeback]
root	37	2	0	01:45	?	00:00:00	[kcompactd0]
root	38	2	0	01:45	?	00:00:00	[ksmd]
root	39	2	0	01:45	?	00:00:00	[khugepaged]
root	40	2	0	01:45	?	00:00:00	[kworker/R-kintegrityd]
root	41	2	0	01:45	?	00:00:00	[kworker/R-kblockd]
root	42	2	0	01:45	?	00:00:00	[kworker/R-blkcg_punt_bio]

root	44	2	0	01:45	?	00:00:00	[irq/9-acpi]
root	45	2	0	01:45	?	00:00:00	[kworker/R-tpm_dev_wq]
root	46	2	0	01:45	?	00:00:00	[kworker/R-ata_sff]
root	47	2	0	01:45	?	00:00:00	[kworker/R-md]
root	48	2	0	01:45	?	00:00:00	[kworker/R-md_bitmap]
root	49	2	0	01:45	?	00:00:00	[kworker/R-edac-poller]
root	50	2	0	01:45	?	00:00:00	[kworker/R-devfreq_wq]
root	51	2	0	01:45	?	00:00:00	[watchdogd]
root	52	2	0	01:45	?	00:00:00	[kworker/0:1H-kblockd]
root	53	2	0	01:45	?	00:00:00	[kswapd0]
root	54	2	0	01:45	?	00:00:00	[ecryptfs-kthread]
root	55	2	0	01:45	?	00:00:00	[kworker/R-kthrotld]
root	56	2	0	01:45	?	00:00:00	[kworker/R-acpi_thermal_pm]
root	57	2	0	01:45	?	00:00:00	[kworker/R-nvme-wq]
root	58	2	0	01:45	?	00:00:00	[kworker/R-nvme-reset-wq]
root	59	2	0	01:45	?	00:00:00	[kworker/R-nvme-delete-wq]
root	60	2	0	01:45	?	00:00:00	[kworker/R-nvme-auth-wq]
root	63	2	0	01:45	?	00:00:00	[kworker/R-mld]
root	64	2	0	01:45	?	00:00:00	[kworker/R-ipv6_addrconf]
root	71	2	0	01:45	?	00:00:00	[kworker/R-kstrp]
root	73	2	0	01:45	?	00:00:00	[kworker/u9:0]
root	86	2	0	01:45	?	00:00:00	[kworker/R-charger_manager]
root	87	2	0	01:45	?	00:00:00	[jbd2/nvme0n1p1-8]
root	88	2	0	01:45	?	00:00:00	[kworker/R-ext4-rsv-conversion]
root	89	2	0	01:45	?	00:00:00	[kworker/1:1H-kblockd]
root	123	2	0	01:45	?	00:00:00	[psimon]
root	128	1	0	01:45	?	00:00:00	/usr/lib/systemd/systemd-journald
root	163	2	0	01:45	?	00:00:00	[kworker/R-kmpathd]
root	164	2	0	01:45	?	00:00:00	[kworker/R-kmpath_handlerd]
root	190	1	0	01:45	?	00:00:00	/sbin/multipathd -d -s
root	196	1	0	01:45	?	00:00:00	/usr/lib/systemd/systemd-udev
root	209	2	0	01:45	?	00:00:00	[psimon]
root	257	2	0	01:45	?	00:00:00	[kworker/R-cryptd]
root	258	2	0	01:45	?	00:00:00	[kworker/R-ena]
root	289	2	0	01:45	?	00:00:00	[jbd2/nvme0n1p16-8]
root	290	2	0	01:45	?	00:00:00	[kworker/R-ext4-rsv-conversion]
991	324	1	0	01:45	?	00:00:00	/usr/lib/systemd/systemd-resolved
998	474	1	0	01:45	?	00:00:00	/usr/lib/systemd/systemd-networkd
root	515	1	0	01:45	?	00:00:00	/usr/sbin/acpid
root	519	1	0	01:45	?	00:00:00	/usr/sbin/cron -f -P

```

root      786      1  0 01:45 ?      00:00:01 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root      1157     1  0 01:46 ?      00:00:00 sshd: /usr/sbin/sshd -D -o AuthorizedKeysCommand /usr/share/ec2-instance-connect/eic_run_authorized_keys
root      1159     1  0 01:46 ?      00:00:00 sshd: ubuntu [priv]
ubuntu    1163     1  0 01:46 ?      00:00:00 /usr/lib/systemd/systemd --user
ubuntu    1164    1163  0 01:46 ?      00:00:00 (sd-pam)
ubuntu    1272    1159  0 01:46 ?      00:00:00 sshd: ubuntu@pts/0
ubuntu    1273    1272  0 01:46 pts/0      00:00:00 -bash
root      1319    1157  0 01:56 ?      00:00:00 sshd: ubuntu [priv]
ubuntu    1375    1319  0 01:56 ?      00:00:02 sshd: ubuntu@pts/1
ubuntu    1376    1375  0 01:56 ?      00:00:00 -bash
root      1874      2  0 02:23 ?      00:00:00 [kworker/u8:4-events_power_efficient]
root      1941    1157  0 02:45 ?      00:00:00 sshd: ubuntu [priv]
ubuntu    1997    1941  0 02:45 ?      00:00:00 sshd: ubuntu@pts/2
ubuntu    1998     2  0 02:45 ?      00:00:00 -bash
root      2016    1997  0 02:45 ?      00:00:00 sudo docker run -it --name con ubuntu bash
root      2017    2016  0 02:46 ?      00:00:00 sudo docker run -it --name con ubuntu bash
root      2018    2017  0 02:46 ?      00:00:00 docker run -it --name con ubuntu bash
root      2035      1  0 02:46 ?      00:00:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 7a6eb5a72a7848fb92f56c8fddbd6004b11f2df8e707302de3a
root      2058    2035  0 02:46 pts/0      00:00:00 bash
root      2110      2  0 02:50 ?      00:00:00 [kworker/1:1-mm_percpu_wq]
root      2142      2  0 02:56 ?      00:00:00 [kworker/0:2]
root      2147      2  0 03:00 ?      00:00:00 [kworker/1:3-cgroup_destroy]
root      2148      2  0 03:00 ?      00:00:00 [kworker/u8:0-events_power_efficient]
root      2157    1376  0 03:07 ?      00:00:00 sudo docker run -it --name con1 --pid=host --network host ubuntu bash
root      2158    2157  0 03:07 ?      00:00:00 sudo docker run -it --name con1 --pid=host --network host ubuntu bash
root      2159    2158  0 03:07 ?      00:00:00 docker run -it --name con1 --pid=host --network host ubuntu bash
root      2166      2  0 03:07 ?      00:00:00 [kworker/u8:1-kvfree_rcu_reclaim]
root      2177      1  0 03:07 ?      00:00:00 /usr/bin/containerd-shim-runc-v2 -namespace moby -id 3876fda8d1c858377906d941b17486196a5230dd8cb6e4e5f22
root      2201    2177  0 03:07 pts/0      00:00:00 bash
root      2215    1157  0 03:07 ?      00:00:00 sshd: ubuntu [priv]
ubuntu    2313    2215  0 03:07 ?      00:00:00 sshd: ubuntu@pts/5
ubuntu    2314    2313  0 03:07 ?      00:00:00 -bash
root      2353    1157  0 03:11 ?      00:00:00 sshd: ubuntu [priv]
ubuntu    2409    2353  0 03:11 ?      00:00:00 sshd: ubuntu@pts/6
ubuntu    2410    2409  0 03:11 ?      00:00:00 -bash
root      2428    2410  0 03:12 ?      00:00:00 sudo docker exec -it con bash
root      2429    2428  0 03:12 ?      00:00:00 sudo docker exec -it con bash
root      2430    2429  0 03:12 ?      00:00:00 docker exec -it con bash
root      2449    2035  0 03:12 ?      00:00:00 bash
root      2461    2201  0 03:12 pts/0      00:00:00 ps -ef

```

Find Name Space

1. Default Container

```

ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con
2058
ubuntu@ip-10-0-1-18:~$ ps -ef | grep [2]058
root      2058      2035  0 02:46 pts/0      00:00:00 bash

ubuntu@ip-10-0-1-18:~$ sudo ls -l /proc/2058/ns/pid
lrwxrwxrwx 1 root root 0 Nov  7 02:48 /proc/2058/ns/pid -> 'pid:[4026532218]'

```

2. Container shared the host ns

```

ubuntu@ip-10-0-1-18:~$ sudo docker inspect --format '{{.State.Pid}}' con1
3518
ubuntu@ip-10-0-1-18:~$ ps -ef | grep [3]518
root      3518      3494  0 03:54 pts/0      00:00:00 bash

ubuntu@ip-10-0-1-18:~$ sudo ls -l /proc/3518/ns/pid
lrwxrwxrwx 1 root root 0 Nov  7 04:04 /proc/3518/ns/pid -> 'pid:[4026531836]'

```

3. Host

```

ubuntu@ip-10-0-1-18:~$ sudo ls -l /proc/1/ns/pid
lrwxrwxrwx 1 root root 0 Nov  7 04:05 /proc/1/ns/pid -> 'pid:[4026531836]'

```


Observation

When we create a default container and list down the process inside that container we only see that only the container's processes.

When we create a container which shares the host's Pid. We list down the process inside the container; actually it shows all the host's processes.